Kazakh-British Technical University

Web Development

Lecture 2 • **HTML & CSS Foundations**

Subject

Course Web Development

Term Spring 2026

Topic HTML & CSS Foundations

Lecturer: `Seitbekov Sanzhar`

Today: semantic HTML, forms, CSS basics, Flexbox/Grid, mini project.

Tip: ←/→ to navigate • H help • P print/scroll mode

KBTU KAZAKH-BRITISH TECHNICAL UNIVERSITY

Web Development • Lecture 2

HTML & CSS: build your first real UI

Today we go from “I saw tags once” → to a clean page layout, a working form, and styling that doesn’t look like 2007.

Goal make a mini profile page

Skill semantic HTML

Skill CSS basics + layout

Bonus animations

What we'll do

Plan for today (2 hours)

Part A — HTML

- How browser reads an HTML document
- Elements vs attributes
- Semantic layout (header/main/footer)
- Forms + input types + validation

> Live demo #1: build a login form that validates itself.

Part B — CSS

- How CSS is applied (cascade)
- Selectors + specificity (the "why it doesn't work" part)
- Box model, spacing, sizing
- Flexbox + Grid (layout superpowers)
- Transitions / animations

> Live demo #2: profile card with hover + animation.

Roadmap

Where lecture 2 fits

HTML

Structure. Meaning. Accessibility. SEO. It's the skeleton.

Build forms, pages

Think semantics

CSS

Look & layout. Responsive design. Animations. It's the skin + motion.

Box model

Layout flex/grid

JavaScript

Behavior. Interactivity. Data. That's next lecture.

DOM events

Fetch APIs

Browser basics

What happens when you open a website?

1. **Request** HTML from a server
2. **Parse HTML** → build DOM tree
3. **Load CSS** → build CSSOM
4. **Combine** DOM + CSSOM → Render Tree
5. **Layout** (sizes/positions)
6. **Paint** pixels on screen

> If you understand DOM + CSSOM, debugging becomes 10× easier.

Mini-wow

Open DevTools → Elements tab → watch DOM change live.

```
Right click → Inspect
Ctrl+Shift+I (Win/Linux)
⌥⌘I (macOS)
```

We'll do this in 60 seconds on the next demo.

HTML

The minimal HTML document

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello, Web!</h1>
  </body>
</html>
```

Rule: everything visible goes inside `<body>`. Metadata + links go inside `<head>`.

HTML

Elements vs attributes

Element

“What it is” — heading, paragraph, link, button...

```
<p>This is a paragraph</p>
<a>This is a link</a>
```

Attribute

“Extra info” — where link goes, image source, input type...

```
<a href="https://example.com">Go</a>
<img src="cat.png" alt="Cat">
<input type="email" required>
```

Semantics

Semantics: why meaning matters

Div soup 😵‍💫

```
<div class="top">
  <div class="menu">...</div>
</div>
<div class="content">...</div>
<div class="bottom">...</div>
```

Works... but search engines & screen readers see “just div”.

Semantic layout 🧠

```
<header>...</header>
<nav>...</nav>
<main>...</main>
<footer>...</footer>
```

> Semantics = accessibility + SEO + future you says thanks.

A Common tags

Text content you'll use daily

Headings

```
<h1>Title</h1>
<h2>Section</h2>
<h3>Subsection</h3>
```

Text

```
<p>Paragraph</p>
<strong>Important</strong>
<em>Emphasis</em>
```

Inline code

```
<code>npm install</code>
<pre>...</pre>
```

Media

Links, images, and paths

Links

```
<a href="https://kbtu.kz">KBTU</a>
<a href="/about.html">About</a>
<a href="#contact">Jump</a>
```

Relative vs absolute paths. “/” is the site root.

Images

```
<img src="images/logo.png" alt="Site logo">
```

> If your page “looks ok” but is inaccessible — missing `alt` is a classic.

Structure

Lists and tables (when appropriate)

Lists

```
<ul>
  <li>Milk</li>
  <li>Bread</li>
</ul>

<ol>
  <li>Install VS Code</li>
  <li>Create index.html</li>
</ol>
```

Tables

```
<table>
  <tr><th>Week</th><th>Topic</th></tr>
  <tr><td>2</td><td>HTML+CSS</td></tr>
</table>
```

Tables are for **data**, not for layout (we have CSS for that).

HTML5

HTML5 semantic layout example

```
<header>
  <h1>My Portfolio</h1>
  <nav>...</nav>
</header>

<main>
  <section>...</section>
  <article>...</article>
</main>

<footer>© 2026</footer>
```

Think: **main** = unique content, **article** = stand-alone piece, **section** = grouped topic.

HTML

Embedding: video / audio / iframe

Video

```
<video controls width="420">
  <source src="demo.mp4" type="video/mp4">
</video>
```

Use `controls` so user can play/pause.

Iframe (careful)

```
<iframe
  src="https://example.com"
  title="Embedded site">
</iframe>
```

> If iframe is blocked → the site uses security headers. That's normal.

Forms

Forms are the bridge: UI → data

Why forms matter

- Login, register, checkout, search...
- Even “like” buttons are basically a form + request
- Later: Angular forms + Django API

> If you can build forms well → you can build real apps.

Form anatomy

```
<form action="/login" method="POST">
  <label>Email</label>
  <input type="email">

  <button type="submit">Sign in</button>
</form>
```

For now we focus on HTML behavior (no backend yet).

Inputs

Input types you should know

text name, title

email validation + mobile keyboard

password hidden text

number min/max

date date picker

checkbox options

radio choose one

file upload

```
<input type="email" placeholder="you@kbtu.kz">
<input type="password" minlength="8">
<input type="date">
```

Validation

Built-in validation (free!)

Useful attributes

```
<input required>
<input minlength="8">
<input maxlength="32">
<input pattern="[A-Za-z]{3,}">
<input min="0" max="100">
```

Why it’s cool

- Less JS needed
- Consistent user experience
- Good baseline for security (but backend still must validate)

> Front-end validation = convenience, not security.

Accessibility

Labels & accessibility: professional habit

Correct

```
<label for="email">Email</label>
<input id="email" type="email">
```

Clicking the label focuses the input (nice UX).

Also useful

```
<button aria-label="Close">×</button>
```

> Accessibility is not “extra”. It’s the default if you want to work in industry.

Live demo

Demo #1: build a login form

Edit the HTML on the left → see result on the right. Then “wow” them by adding `:invalid` CSS later.

HTML

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
  <title>Login</title>
  <style>
    body{font-family:system-ui; padding:24px;
background:#0b1020; color:#fff;}
    .card{max-width:420px; margin:auto; padding:18px; border-
radius:16px; background:#121a33;}
    label{display:block; margin-top:12px; opacity:.85}
    input{width:100%; padding:10px 12px; margin-top:6px;
```

Run Reset Try: add `pattern` to password, change placeholders.

Preview

Pro tip: submit empty → see browser validation.

CSS

3 ways to add CSS (pick wisely)

External (recommended)

```
<link rel="stylesheet" href="styles.css">
```

Scales well. Keeps HTML clean.

Internal / inline (use rarely)

```
<style> h1{color:red} </style>
<h1 style="color:red">...</h1>
```

> Inline styles are the “fast food” of CSS: quick but not healthy long-term.

> Meme: “inline styles” vs “external CSS”. (Wikimedia Commons)

CSS

Selectors: how CSS chooses elements

Common selectors

```
p { ... }            /* tag */
.card { ... }        /* class */
#main { ... }        /* id */
.card p { ... }      /* descendant */
button:hover { ... } /* pseudo-class */
```

Rule format

```
selector {
  property: value;
  property: value;
}
```

> When CSS “doesn’t work”, 80% is selector not matching what you think.

Cascade

Cascade & specificity (the boss fight)

Specificity score

```
inline style      1000
#id               100
.class / :hover    10
tag / ::before      1

Example:
div.card = 11
```

If two rules match → more specific wins. If equal → later wins.

Debug trick

In DevTools → “Styles” pane shows crossed-out rules (they lost).

> Teach students to trust DevTools more than guesses.

CSS

Units & colors

Units

```
px   /* pixels */
%    /* relative to parent */
rem  /* relative to root font-size */
vh/vw/* viewport units */
```

Try to prefer `rem` for typography.

Colors

```
color: #ff0066;
background: rgb(0 0 0 / 60%);
background: hsl(200 90% 55%);
```

> HSL is great when you want to tweak “same color, different brightness”.

CSS

Box model: why spacing is weird

Every element is a box

```
content
padding
border
margin
```

Margin is outside, padding is inside.

Pro move

```
* { box-sizing: border-box; }
```

> This makes width/height include padding/border — fewer surprises.

CSS

Display: block vs inline vs inline-block

Block

Starts new line, takes full width by default.

```
div, p, section
```

Inline

Sits inside line, width/height ignored.

```
span, a, strong
```

> Inline elements don’t accept vertical margin like you expect.

↔ Layout

Flexbox: 80% of modern layout

```
.row {
  display: flex;
  gap: 12px;
  justify-content: space-between;
  align-items: center;
}
```

When to use

- Navigation bar
- Buttons in a row
- Centering a card

Mindset

Flexbox is 1D (row OR column). Grid is 2D.

> If you only learn one layout system today: Flexbox.

Layout

CSS Grid: layouts that feel like magic

```
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 16px;
}
```

> Grid is perfect for page-level layout: sidebars, dashboards, galleries.

Position

Positioning: relative / absolute / fixed / sticky

```
.parent { position: relative; }
.child  { position: absolute; top: 10px; right: 10px; }
```

Absolute positions relative to nearest positioned parent.

```
.fab { position: fixed; bottom: 16px; right: 16px; }
```

Sticky is great for headers: `position: sticky; top: 0;`

Responsive

Responsive design: one page, many screens

```
@media (max-width: 720px) {
  .grid { grid-template-columns: 1fr; }
}
```

2 rules to remember

- Use fluid layouts (%, fr, flex)
- Use breakpoints for big changes

Instant wow

Open DevTools → Toggle device toolbar → resize live.

> Students love seeing the page “transform” on screen.

CSS motion

Demo #2: hover + transition + micro-animation

HTML + CSS

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
  <title>Profile Card</title>
  <style>
    :root{ --bg:#0b1020; --card:#121a33; --c1:#00f3ff; --
c2:#bc13fe; --c3:#00ff9d; }
    body{ margin:0; font-family:system-ui; background: radial-
gradient(circle at 20% 10%, #17214a, var(--bg)); color:#fff;
          min-height:100vh; display:grid; place-items:center;
padding:24px; }
    .card{ width:min(520px, 92vw); background:linear
```

Run Reset Try: change colors, chip text, add an emoji ✨

Preview

Hover the card → motion + glow.

Mini project

Mini project: one-page profile

Must-have

- Semantic layout: header / main / footer
- Profile card with avatar + text
- Links: GitHub, Telegram, email
- Responsive: looks OK on phone

“Wow” options

- Hover animation on card
- Gradient border / glow
- Form with validation
- Dark/light mode toggle (optional)

> Your goal is not perfection — it’s building taste + confidence.

Q & A

Ask anything — we'll debug it together.

Contact & Info

How to reach me

Seitbekov Sanzhar

Lecturer

s.seitbekov@kbtu.kz

Office 178

Course chat

Microsoft Teams

Team code: `ksg13tw`

- Announcements + materials
- Homework questions
- Useful links (cheatsheets, docs)